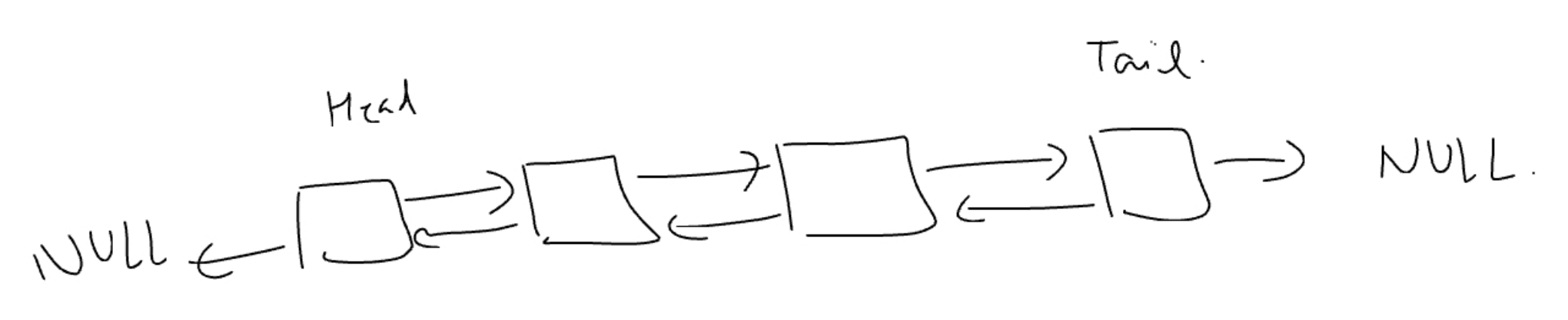
Deque.

push_back() push_front()
pop_back() pop_front()
back() front()



Deques can be implemented using:
1) Static Arrays.
2) Doubly linked list

[1, 2, 3, 1, 4, 5, 2, 3, 6] K = 3.

[3, 3, 4, 5, 5, 5, 6]

0 1 2 3 4 5 6 7 8
[1, 2, 3, 1, 4, 5, 2, 3, 24]

mx = ~~3~~ ~~4~~ 5

[3, 3, 4, 5, 5, 5, 5

BF.

TC = O(nk)
AS = O(1).

$i: 0 \to (n-k)$
$mx = -\infty$
$j: 0 \to k-1$
$mx = \max(mx, arr[i+j])$.

res[i] = mx.

[8, 5, 10, 7, 9, 4, 95, 12, 90, 13] K = 4.

[10, 10, 10, 9, 12, 90, 90]

Deque.
~~8~~, ~~5~~ ~~10~~, ~~7~~ ~~9~~, ~~4~~, ~~9~~
~~12~~ 90, 13

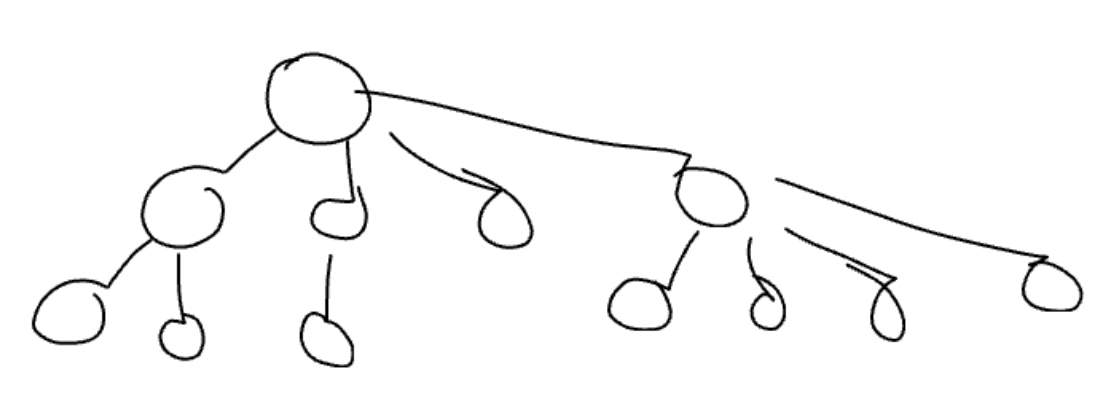
TC:

Deque.
push_back()
pop_back()
front()
pop_front()

4 [5 6 7 8] 9 . . . .] K = 4

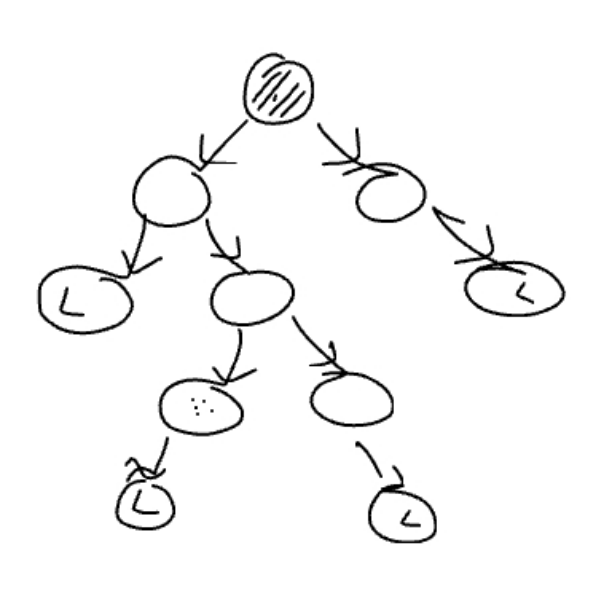
[- . . . .

TREE.



```
class Node {
    int data.
    vector<Node*> children;
}
```

BINARY TREE

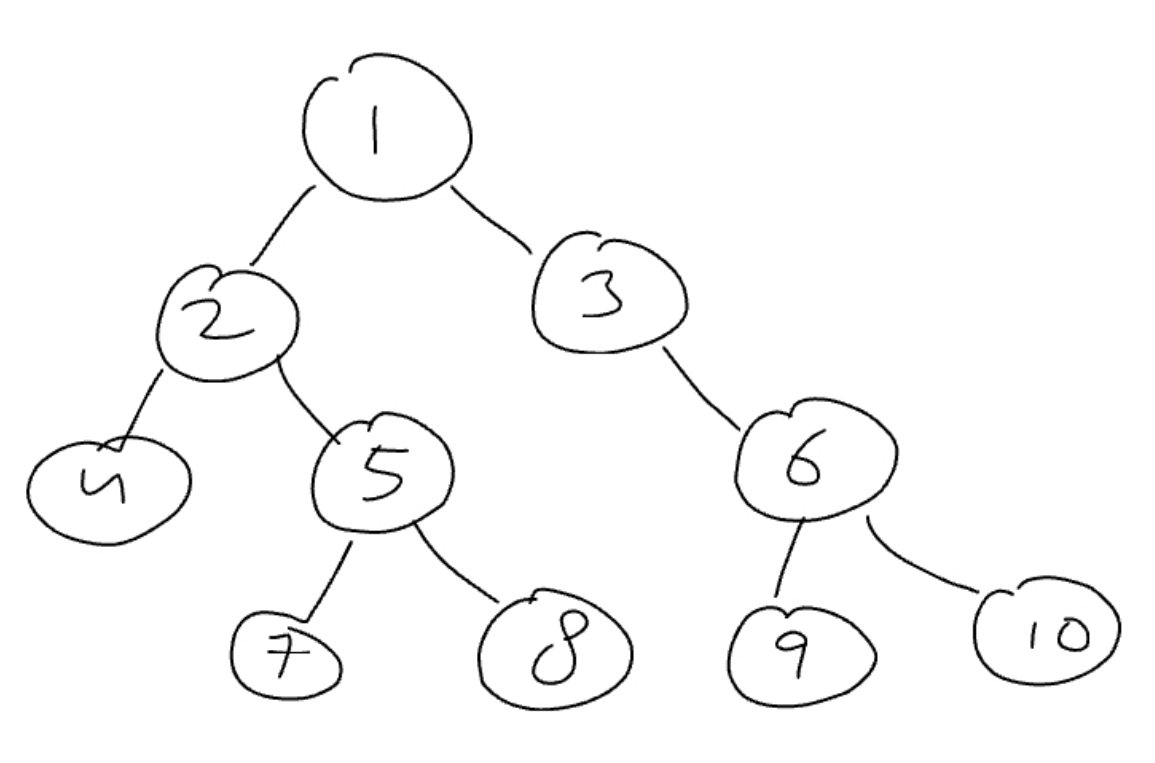
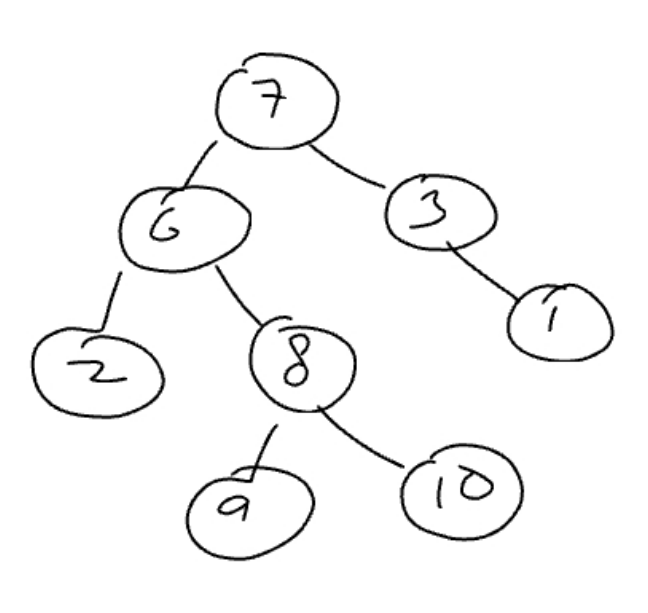


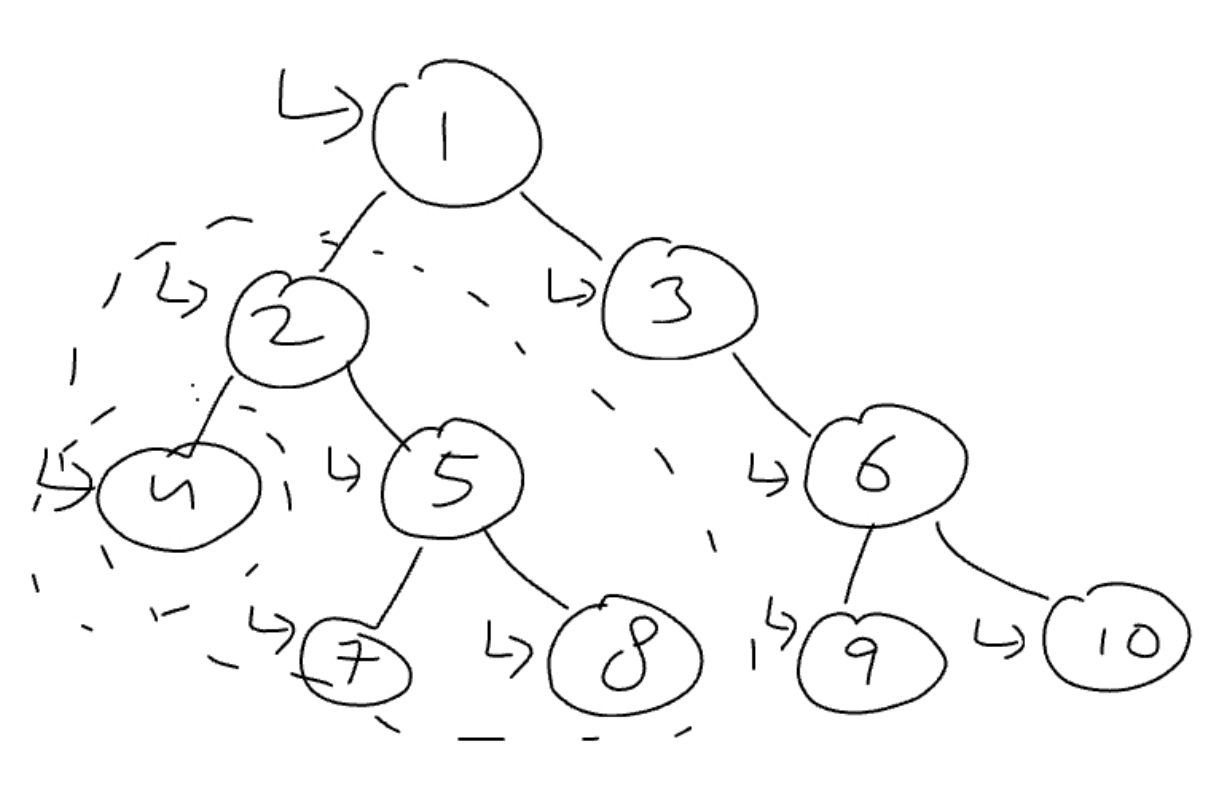
```
class Node {
    int data;
    Node* left;
    Node* right;
}
```

Tree Traversals

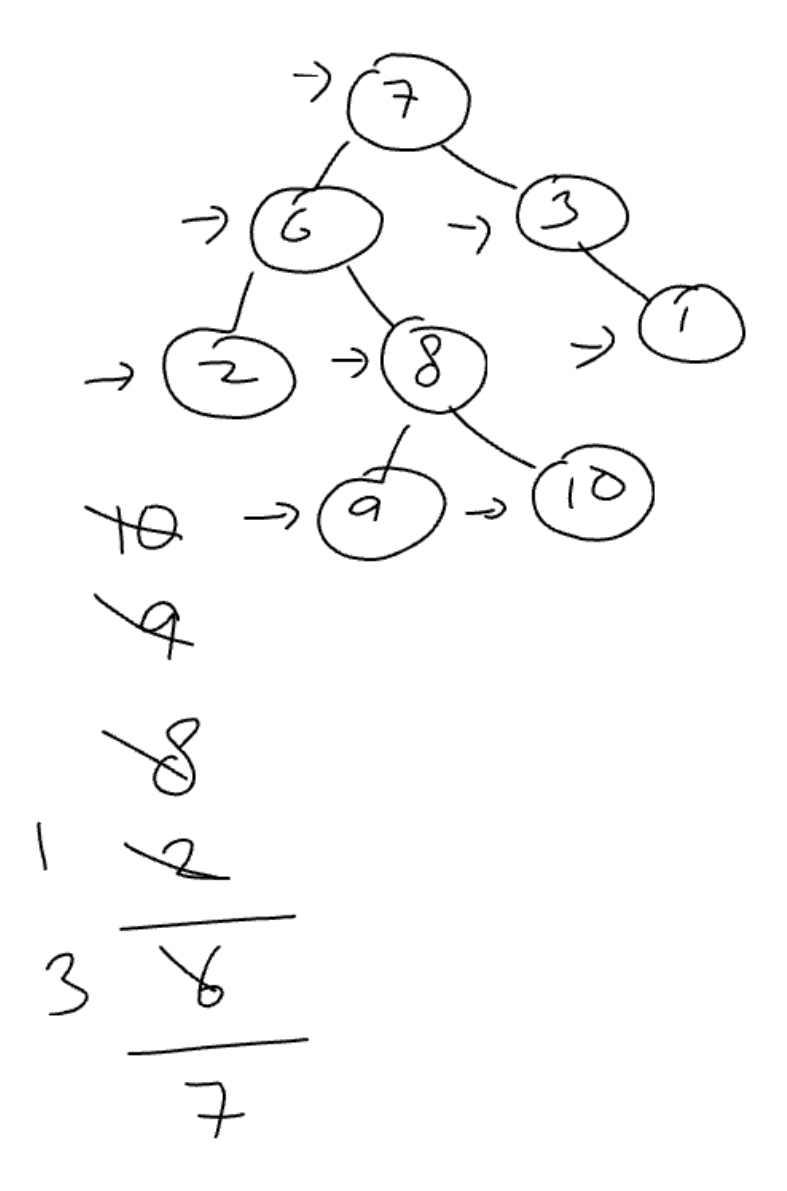
Preorder.

1) Visit the root
2) Preorder traversal of left subtree
3) " " " right subtree.



[1, 2, 4, 5, 7, 8, 3, 6, 9, 10]



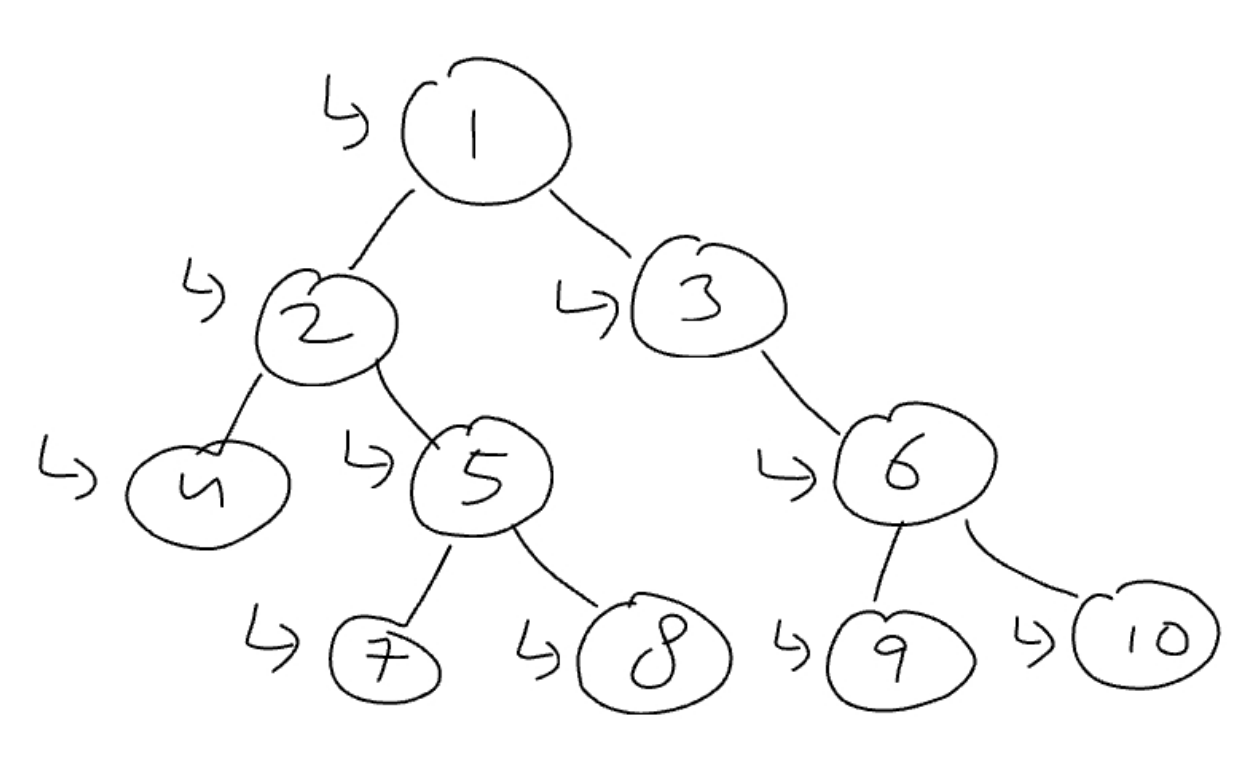
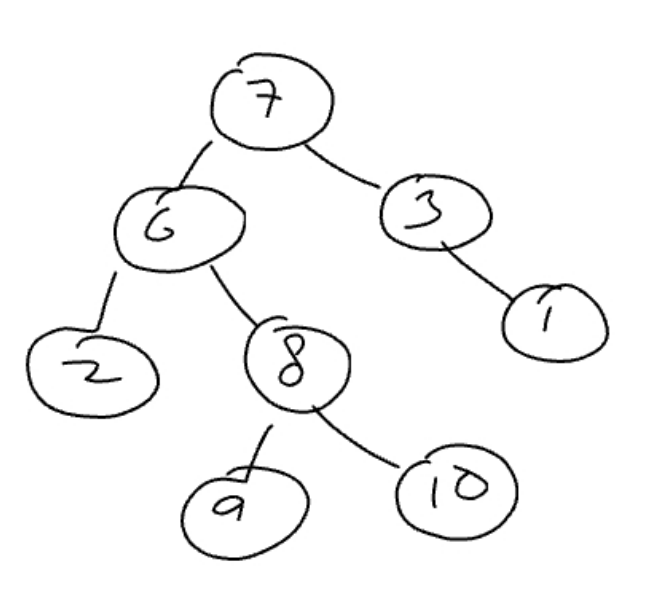
[7, 6, 2, 8, 9, 10, 3, 1]

TC = O(n)
AS = O(H).

Postorder.

1) Left
2) Right
3) Self.

Rec.

[4, 7, 8, 5, 2, 9, 10, 6, 3, 1]

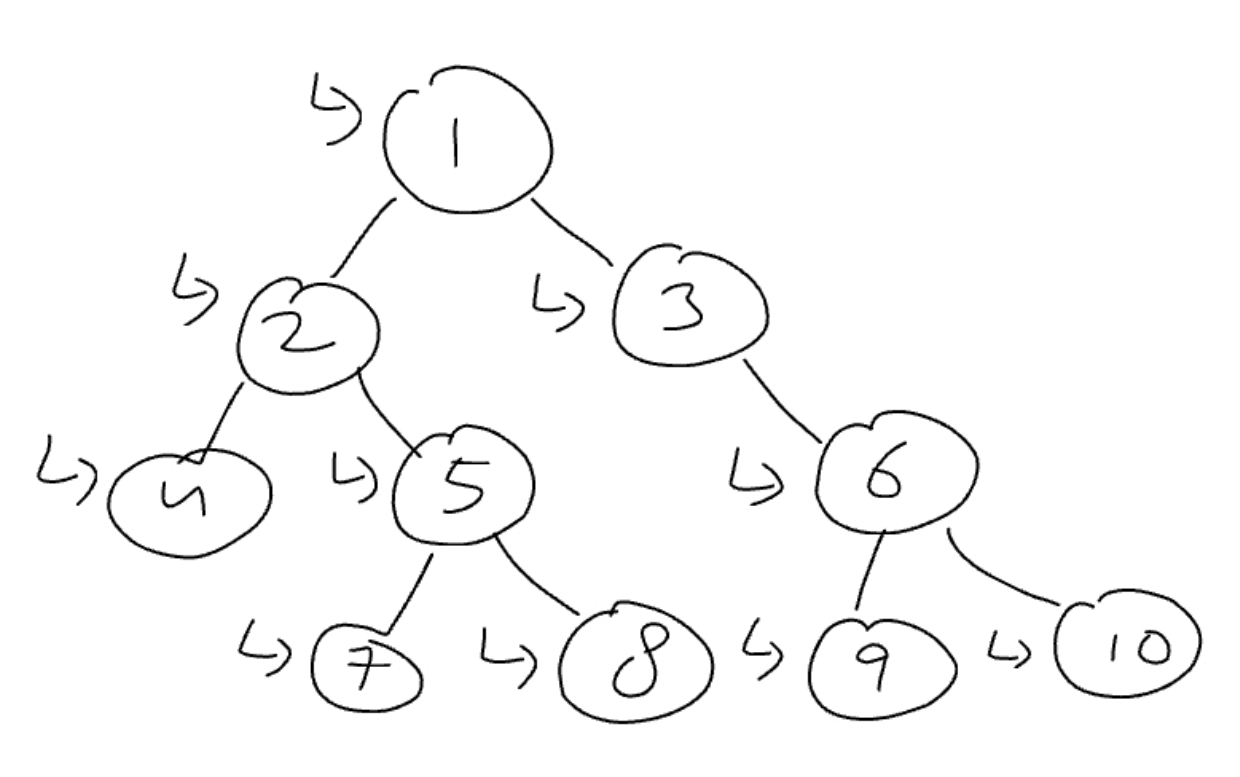
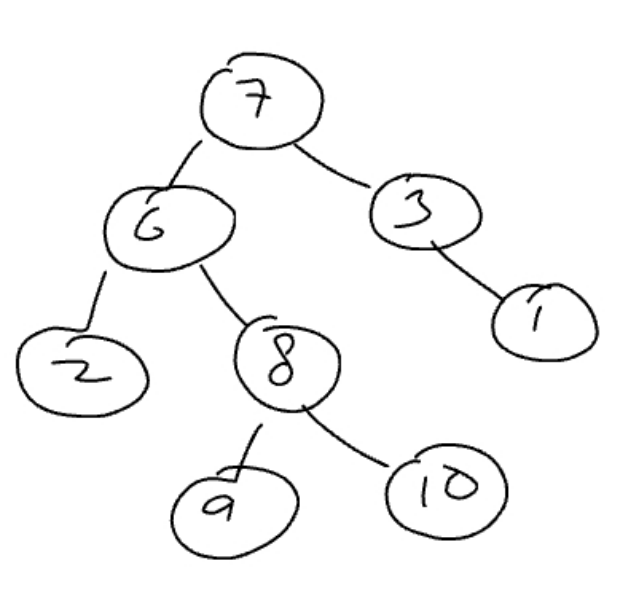
[2, 9, 10, 8, 6, 1, 3, 7].

TC = O(n)
AS = O(H).

Inorder

1) Left
2) Self
3) Right

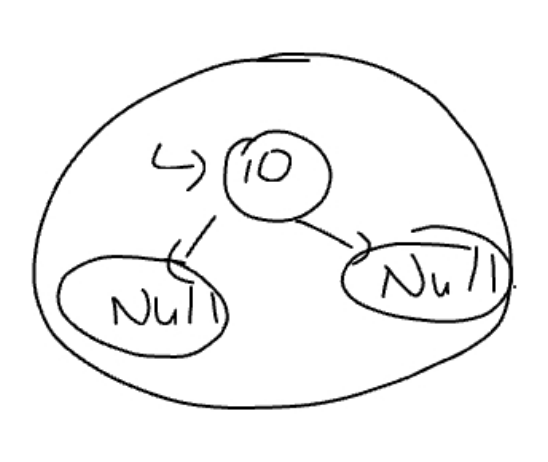
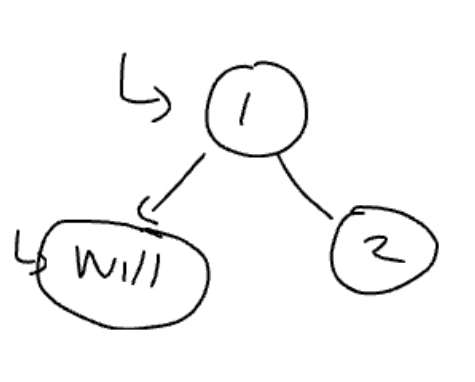
→ Rec.

[4, 2, 7, 5, 8, 1, 3, 9, 6, 10]

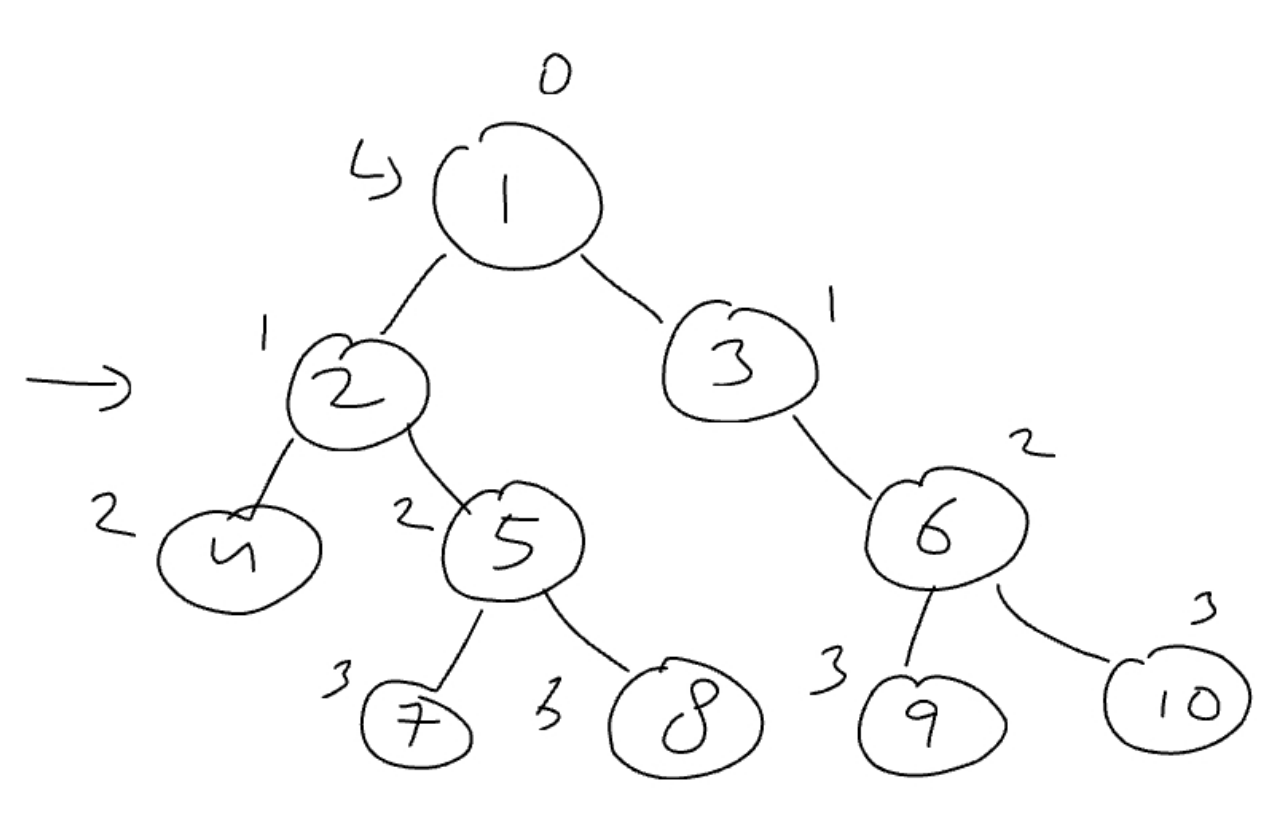
[2, 6, 9, 8, 10, 7, 3, 1]

TC = O(n)
AS = O(H).

1

Levelorder Traversals



[1, 2, 3, 4, 5, 6, 7, 8, 9, 10].

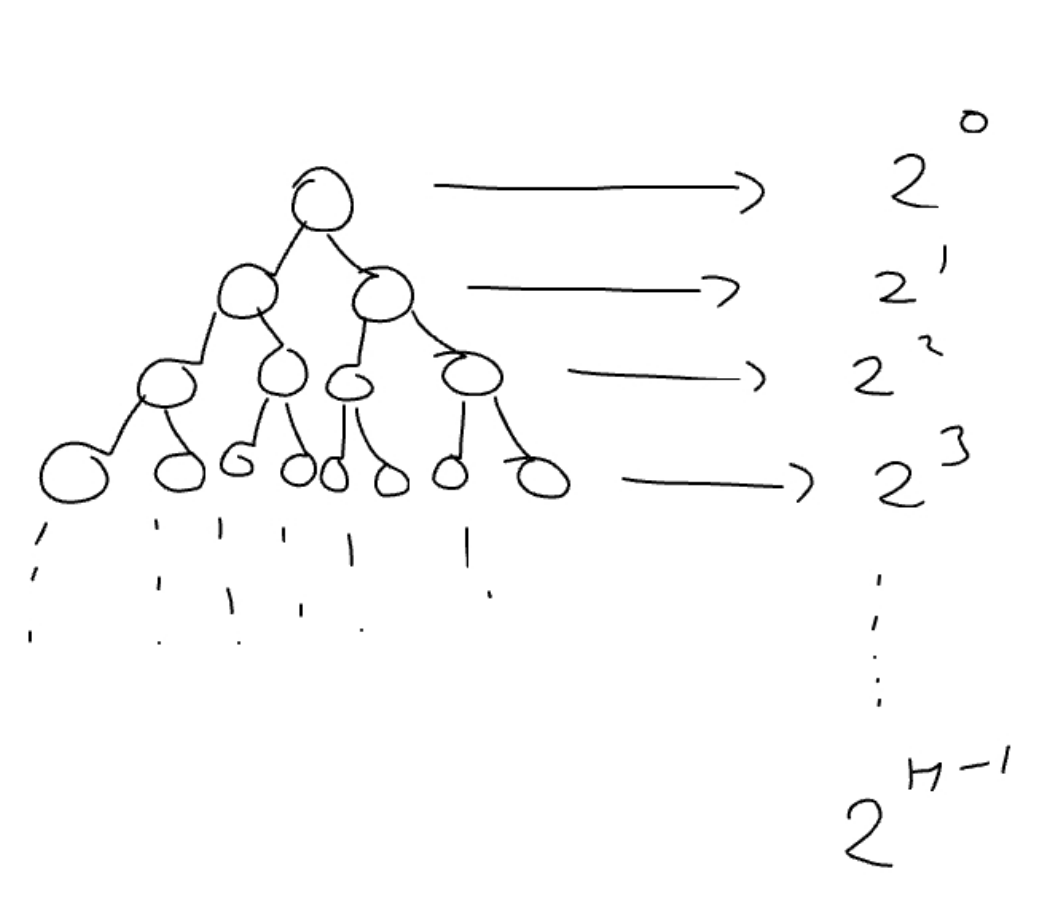
int → [nodes at that level].
(level)

~~1~~, ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~, ~~7~~, ~~8~~, ~~9~~, ~~10~~

0 → [1
1 → [2, 3
2 → [4, 5, 6
3 → [7, 8, 9, 10]

[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

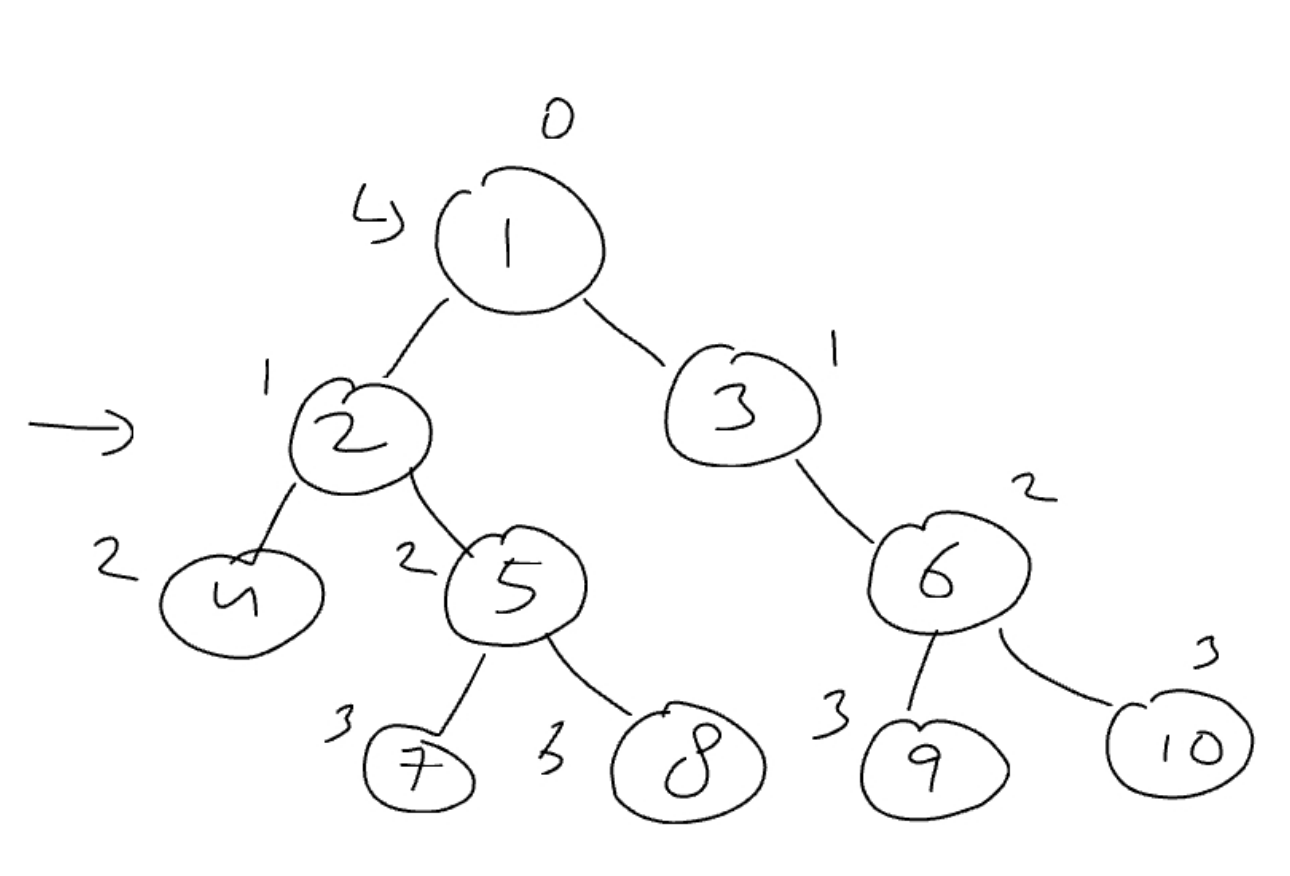
TC = O(n)
AS = O(n)



$$O(2^{H-1})$$
$$= O(2^{\log_2 n - 1})$$
$$= O\left(\frac{2^{\log_2 n}}{2}\right)$$
$$= O\left(\frac{n}{2}\right) = O(n).$$

Level-by-level ~~LOT~~.
Level order Trav.

[[1]
[2, 3]
[4, 5, 6]
[7, 8, 9, 10]]



Q: ~~1~~ ~~2~~, ~~3~~, ~~4~~, ~~5~~, ~~6~~. ~~7~~, 8, ~~9~~, ~~10~~

sz = ~~1~~ ~~2~~ ~~3~~ 4

res: [[1],
[2, 3]
[4, 5, 6]
[7, 8, 9, 10]].